



Compiler

Copyright © 1997-2026 Ericsson AB. All Rights Reserved.
Compiler 8.4.3.4
May 27, 2026

Copyright © 1997-2026 Ericsson AB. All Rights Reserved.

Licensed under the Apache License, Version 2.0 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0> Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License. Ericsson AB. All Rights Reserved..

May 27, 2026

1 Reference Manual

The Compiler application compiles Erlang code to byte-code. The highly compact byte-code is executed by the Erlang emulator.

compile

Erlang module

This module provides an interface to the standard Erlang compiler. It can generate either a new file, which contains the object code, or return a binary, which can be loaded directly.

Data Types

`option() = term()`

See `file/2` for detailed description

`forms() = term()`

List of Erlang abstract or Core Erlang format representations, as used by `forms/2`

Exports

`env_compiler_options()`

Return compiler options given via the environment variable `ERL_COMPILER_OPTIONS`. If the value is a list, it is returned as is. If it is not a list, it is put into a list.

`file(File)`

Is the same as `file(File, [verbose, report_errors, report_warnings])`.

`file(File, Options) -> CompRet`

Types:

`CompRet = ModRet | BinRet | ErrRet`

`ModRet = {ok, ModuleName} | {ok, ModuleName, Warnings}`

`BinRet = {ok, ModuleName, Binary} | {ok, ModuleName, Binary, Warnings}`

`ErrRet = error | {error, Errors, Warnings}`

Compiles the code in the file `File`, which is an Erlang source code file without the `.erl` extension. `Options` determine the behavior of the compiler.

Returns `{ok, ModuleName}` if successful, or `error` if there are errors. An object code file is created if the compilation succeeds without errors. It is considered to be an error if the module name in the source code is not the same as the basename of the output file.

Available options:

`brief`

Restricts error and warning messages to a single line of output. As of OTP 24, the compiler will by default also display the part of the source code that the message refers to.

`basic_validation`

This option is a fast way to test whether a module will compile successfully. This is useful for code generators that want to verify the code that they emit. No code is generated. If warnings are enabled, warnings generated by the `erl_lint` module (such as warnings for unused variables and functions) are also returned.

Use option `strong_validation` to generate all warnings that the compiler would generate.

strong_validation

Similar to option `basic_validation`. No code is generated, but more compiler passes are run to ensure that warnings generated by the optimization passes are generated (such as clauses that will not match, or expressions that are guaranteed to fail with an exception at runtime).

binary

The compiler returns the object code in a binary instead of creating an object file. If successful, the compiler returns `{ok, ModuleName, Binary}`.

bin_opt_info

The compiler will emit informational warnings about binary matching optimizations (both successful and unsuccessful). For more information, see the section about `bin_opt_info` in the Efficiency Guide.

`{compile_info, [{atom(), term()}]}`

Allows compilers built on top of `compile` to attach extra compilation metadata to the `compile_info` chunk in the generated beam file.

It is advised for compilers to remove all non-deterministic information if the `deterministic` option is supported and it was supplied by the user.

compressed

The compiler will compress the generated object code, which can be useful for embedded systems.

debug_info

Includes debug information in the form of Erlang Abstract Format in the `debug_info` chunk of the compiled beam module. Tools such as Debugger, Xref, and Cover require the debug information to be included.

Warning: Source code can be reconstructed from the debug information. Use encrypted debug information (`encrypt_debug_info`) to prevent this.

For details, see `beam_lib(3)`.

`{debug_info, {Backend, Data}}`

Includes custom debug information in the form of a `Backend` module with custom `Data` in the compiled beam module. The given module must implement a `debug_info/4` function and is responsible for generating different code representations, as described in the `debug_info` under `beam_lib(3)`.

Warning: Source code can be reconstructed from the debug information. Use encrypted debug information (`encrypt_debug_info`) to prevent this.

`{debug_info_key, KeyString}`

`{debug_info_key, {Mode, KeyString}}`

Includes debug information, but encrypts it so that it cannot be accessed without supplying the key. (To give option `debug_info` as well is allowed, but not necessary.) Using this option is a good way to always have the debug information available during testing, yet protecting the source code.

`Mode` is the type of crypto algorithm to be used for encrypting the debug information. The default (and currently the only) type is `des3_cbc`.

For details, see `beam_lib(3)`.

encrypt_debug_info

Similar to the `debug_info_key` option, but the key is read from an `.erlang.crypt` file.

For details, see `beam_lib(3)`.

deterministic

Omit the `options` and `source` tuples in the list returned by `Module:module_info(compile)`, and reduce the paths in stack traces to the module name alone. This option will make it easier to achieve reproducible builds.

{feature, Feature, enable | disable}

Enable (disable) the feature `Feature` during compilation. The special feature `all` can be used to enable (disable) all features.

Note:

This option has no effect when used in a `-compile(..)` attribute. Instead, the `-feature(..)` directive (below) should be used.

A feature can also be enabled (disabled) using the `-feature(Feature, enable | disable).module` directive. Note that this directive can only be present in a prefix of the file, before exports and function definitions. This is the preferred method of enabling and disabling features, since it is a local property of a module.

makedep

Produces a Makefile rule to track headers dependencies. No object file is produced.

By default, this rule is written to `<File>.Pbeam`. However, if option `binary` is set, nothing is written and the rule is returned in `Binary`.

The output will be encoded in UTF-8.

For example, if you have the following module:

```
-module(module).  
  
-include_lib("eunit/include/eunit.hrl").  
-include("header.hrl").
```

The Makefile rule generated by this option looks as follows:

```
module.beam: module.erl \  
    /usr/local/lib/erlang/lib/eunit/include/eunit.hrl \  
    header.hrl
```

makedep_side_effect

The dependencies are created as a side effect to the normal compilation process. This means that the object file will also be produced. This option override the `makedep` option.

{makedep_output, Output}

Writes generated rules to `Output` instead of the default `<File>.Pbeam`. `Output` can be a filename or an `io_device()`. To write to stdout, use `standard_io`. However, if `binary` is set, nothing is written to `Output` and the result is returned to the caller with `{ok, ModuleName, Binary}`.

{makedep_target, Target}

Changes the name of the rule emitted to `Target`.

makedep_quote_target

Characters in `Target` special to `make(1)` are quoted.

makedep_add_missing

Considers missing headers as generated files and adds them to the dependencies.

makedep_phony

Adds a phony target for each dependency.

'P'

Produces a listing of the parsed code, after preprocessing and parse transforms, in the file `<File>.P`. No object file is produced.

'E'

Produces a listing of the code, after all source code transformations have been performed, in the file `<File>.E`. No object file is produced.

'S'

Produces a listing of the assembler code in the file `<File>.S`. No object file is produced.

recv_opt_info

The compiler will emit informational warnings about selective receive optimizations (both successful and unsuccessful). For more information, see the section about selective receive optimization in the Efficiency Guide.

report_errors/report_warnings

Causes errors/warnings to be printed as they occur.

report

A short form for both `report_errors` and `report_warnings`.

return_errors

If this flag is set, `{error, ErrorList, WarningList}` is returned when there are errors.

return_warnings

If this flag is set, an extra field, containing `WarningList`, is added to the tuples returned on success.

warnings_as_errors

Causes warnings to be treated as errors. This option is supported since R13B04.

{error_location, line | column}

If the value of this flag is `line`, the location `ErrorLocation` of warnings and errors is a line number. If the value is `column`, `ErrorLocation` includes both a line number and a column number. Default is `column`. This option is supported since Erlang/OTP 24.0.

If the value of this flag is `column`, debug information includes column information.

return

A short form for both `return_errors` and `return_warnings`.

verbose

Causes more verbose information from the compiler, describing what it is doing.

{source, FileName}

Overrides the source file name as presented in `module_info(compile)` and stack traces.

absolute_source

Turns the source file name (as presented in `module_info(compile)` and stack traces) into an absolute path, which helps external tools like `perf` and `gdb` find Erlang source code.

`{outdir, Dir}`

Sets a new directory for the object code. The current directory is used for output, except when a directory has been specified with this option.

`export_all`

Causes all functions in the module to be exported.

`{i, Dir}`

Adds `Dir` to the list of directories to be searched when including a file. When encountering an `-include` or `-include_lib` directive, the compiler searches for header files in the following directories:

- `"."`, the current working directory of the file server
- The base name of the compiled file
- The directories specified using option `i`; the directory specified last is searched first

`{d, Macro}`

`{d, Macro, Value}`

Defines a macro `Macro` to have the value `Value`. `Macro` is of type atom, and `Value` can be any term. The default `Value` is `true`.

`{parse_transform, Module}`

Causes the parse transformation function `Module:parse_transform/2` to be applied to the parsed code before the code is checked for errors.

`from_abstr`

The input file is expected to contain Erlang terms representing forms in abstract format (default file suffix `".abstr"`). Note that the format of such terms can change between releases.

See also the `no_lint` option.

`from_asm`

The input file is expected to be assembler code (default file suffix `".S"`). Notice that the format of assembler files is not documented, and can change between releases.

`from_core`

The input file is expected to be core code (default file suffix `".core"`). Notice that the format of core files is not documented, and can change between releases.

`no_spawn_compiler_process`

By default, all code is compiled in a separate process which is terminated at the end of compilation. However, some tools, like Dialyzer or compilers for other BEAM languages, may already manage their own worker processes and spawning an extra process may slow the compilation down. In such scenarios, you can pass this option to stop the compiler from spawning an additional process.

`no_strict_record_tests`

This option is not recommended.

By default, the generated code for operation `Record#record_tag.field` verifies that the tuple `Record` has the correct size for the record, and that the first element is the tag `record_tag`. Use this option to omit the verification code.

`no_error_module_mismatch`

Normally the compiler verifies that the module name given in the source code is the same as the base name of the output file and refuses to generate an output file if there is a mismatch. If you have a good reason (or other

reason) for having a module name unrelated to the name of the output file, this option disables that verification (there will not even be a warning if there is a mismatch).

`{no_auto_import, [{F,A}, ...]}`

Makes the function `F/A` no longer being auto-imported from the `erlang` module, which resolves BIF name clashes. This option must be used to resolve name clashes with BIFs auto-imported before R14A, if it is needed to call the local function with the same name as an auto-imported BIF without module prefix.

Note:

As from R14A and forward, the compiler resolves calls without module prefix to local or imported functions before trying with auto-imported BIFs. If the BIF is to be called, use the `erlang` module prefix in the call, not `{no_auto_import, [{F,A}, ...]}`.

If this option is written in the source code, as a `-compile` directive, the syntax `F/A` can be used instead of `{F,A}`, for example:

```
-compile({no_auto_import,[error/1]}).
```

`no_auto_import`

Do not auto-import any functions from `erlang` module.

`no_line_info`

Omits line number information to produce a slightly smaller output file.

`no_lint`

Skips the pass that checks for errors and warnings. Only applicable together with the `from_abstr` option. This is mainly for implementations of other languages on top of Erlang, which have already done their own checks to guarantee correctness of the code.

Caveat: When this option is used, there are no guarantees that the code output by the compiler is correct and safe to use. The responsibility for correctness lies on the code or person generating the abstract format. If the code contains errors, the compiler may crash or produce unsafe code.

`{extra_chunks, [{binary(), binary()}]}`

Pass extra chunks to be stored in the `.beam` file. The extra chunks must be a list of tuples with a four byte binary as chunk name followed by a binary with the chunk contents. See `beam_lib` for more information.

`{check_ssa, Tag :: atom()}`

Parse and check assertions on the structure and content of the BEAM SSA code produced by the compiler. The `Tag` indicates the set of assertions to check and after which compiler pass the check is performed. This option is internal to the compiler and can be changed or removed at any time without prior warning.

If warnings are turned on (option `report_warnings` described earlier), the following options control what type of warnings that are generated. Except from `{warn_format, Verbosity}`, the following options have two forms:

- A `warn_xxx` form, to turn on the warning.
- A `nowarn_xxx` form, to turn off the warning.

In the descriptions that follow, the form that is used to change the default value are listed.

`{warn_format, Verbosity}`

Causes warnings to be emitted for malformed format strings as arguments to `io:format` and similar functions. `Verbosity` selects the number of warnings:

- 0 = No warnings

- 1 = Warnings for invalid format strings and incorrect number of arguments
- 2 = Warnings also when the validity cannot be checked, for example, when the format string argument is a variable.

The default verbosity is 1. Verbosity 0 can also be selected by option `nowarn_format`.

`nowarn_bif_clash`

This option is removed, it generates a fatal error if used.

Warning:

As from beginning with R14A, the compiler no longer calls the auto-imported BIF if the name clashes with a local or explicitly imported function, and a call without explicit module name is issued. Instead, the local or imported function is called. Still accepting `nowarn_bif_clash` would make a module calling functions clashing with auto-imported BIFs compile with both the old and new compilers, but with completely different semantics. This is why the option is removed.

The use of this option has always been discouraged. As from R14A, it is an error to use it.

To resolve BIF clashes, use explicit module names or the `{no_auto_import, [F/A]}` compiler directive.

`{nowarn_bif_clash, FAS}`

This option is removed, it generates a fatal error if used.

Warning:

The use of this option has always been discouraged. As from R14A, it is an error to use it.

To resolve BIF clashes, use explicit module names or the `{no_auto_import, [F/A]}` compiler directive.

`nowarn_export_all`

Turns off warnings for uses of the `export_all` option. Default is to emit a warning if option `export_all` is also given.

`warn_export_vars`

Emits warnings for all implicitly exported variables referred to after the primitives where they were first defined. By default, the compiler only emits warnings for exported variables referred to in a pattern.

`nowarn_shadow_vars`

Turns off warnings for "fresh" variables in functional objects or list comprehensions with the same name as some already defined variable. Default is to emit warnings for such variables.

`warn_keywords`

Emits warnings when the code contains atoms that are used as keywords in some feature. When the feature is enabled, any occurrences will lead to a syntax error. To prevent this, the atom has to be renamed or quoted.

`nowarn_unused_function`

Turns off warnings for unused local functions. Default is to emit warnings for all local functions that are not called directly or indirectly by an exported function. The compiler does not include unused local functions in the generated beam file, but the warning is still useful to keep the source code cleaner.

`{nowarn_unused_function, FAS}`

Turns off warnings for unused local functions like `nowarn_unused_function` does, but only for the mentioned local functions. FAS is a tuple `{Name, Arity}` or a list of such tuples.

`nowarn_deprecated_function`

Turns off warnings for calls to deprecated functions. Default is to emit warnings for every call to a function known by the compiler to be deprecated. Notice that the compiler does not know about attribute `-deprecated()`, but uses an assembled list of deprecated functions in Erlang/OTP. To do a more general check, the Xref tool can be used. See also `xref(3)` and the function `xref:m/1`, also accessible through the function `c:xm/1`.

`{nowarn_deprecated_function, MFAs}`

Turns off warnings for calls to deprecated functions like `nowarn_deprecated_function` does, but only for the mentioned functions. `MFAs` is a tuple `{Module, Name, Arity}` or a list of such tuples.

`nowarn_deprecated_type`

Turns off warnings for use of deprecated types. Default is to emit warnings for every use of a type known by the compiler to be deprecated.

`nowarn_removed`

Turns off warnings for calls to functions that have been removed. Default is to emit warnings for every call to a function known by the compiler to have been recently removed from Erlang/OTP.

`{nowarn_removed, ModulesOrMFAs}`

Turns off warnings for calls to modules or functions that have been removed. Default is to emit warnings for every call to a function known by the compiler to have been recently removed from Erlang/OTP.

`nowarn_obsolete_guard`

Turns off warnings for calls to old type testing BIFs, such as `pid/1` and `list/1`. See the Erlang Reference Manual for a complete list of type testing BIFs and their old equivalents. Default is to emit warnings for calls to old type testing BIFs.

`warn_unused_import`

Emits warnings for unused imported functions. Default is to emit no warnings for unused imported functions.

`nowarn_underscore_match`

By default, warnings are emitted when a variable that begins with an underscore is matched after being bound. Use this option to turn off this kind of warning.

`nowarn_unused_vars`

By default, warnings are emitted for unused variables, except for variables beginning with an underscore ("Prolog style warnings"). Use this option to turn off this kind of warning.

`nowarn_unused_record`

Turns off warnings for unused record definitions. Default is to emit warnings for unused locally defined records.

`{nowarn_unused_record, RecordNames}`

Turns off warnings for unused record definitions. Default is to emit warnings for unused locally defined records.

`nowarn_unused_type`

Turns off warnings for unused type declarations. Default is to emit warnings for unused local type declarations.

`nowarn_nif_inline`

By default, warnings are emitted when inlining is enabled in a module that may load NIFs, as the compiler may inline NIF fallbacks by accident. Use this option to turn off this kind of warnings.

`warn_missing_spec`

By default, warnings are not emitted when a specification (or contract) for an exported function is not given. Use this option to turn on this kind of warning.

warn_missing_spec_all

By default, warnings are not emitted when a specification (or contract) for an exported or unexported function is not given. Use this option to turn on this kind of warning.

nowarn_redefined_builtin_type

By default, a warning is emitted when a built-in type is locally redefined. Use this option to turn off this kind of warning.

{nowarn_redefined_builtin_type, Types}

By default, a warning is emitted when a built-in type is locally redefined. Use this option to turn off this kind of warning for the types in **Types**, where **Types** is a tuple **{TypeName, Arity}** or a list of such tuples.

Other kinds of warnings are **opportunistic warnings**. They are generated when the compiler happens to notice potential issues during optimization and code generation.

Note:

The compiler does not warn for expressions that it does not attempt to optimize. For example, the compiler will emit a warning for `1/0` but not for `X/0`, because `1/0` is a constant expression that the compiler will attempt to evaluate. The absence of warnings does not mean that there are no remaining errors in the code.

Opportunistic warnings can be disabled using the following options:

nowarn_opportunistic

Disable all opportunistic warnings.

nowarn_failed

Disable warnings for expressions that will always fail (such as `atom+42`).

nowarn_ignored

Disable warnings for expressions whose values are ignored.

nowarn_nomatch

Disable warnings for patterns that will never match (such as `a=b`) and for guards that always evaluate to `false`.

Note:

All options, except the include path (`{i,Dir}`), can also be given in the file with attribute `-compile([Option,...])`. Attribute `-compile()` is allowed after the function definitions.

Note:

Before OTP 22, the option `{nowarn_deprecated_function, MFAs}` was only recognized when given in the file with attribute `-compile()`. (The option `{nowarn_unused_function, FAs}` was incorrectly documented to only work in a file, but it also worked when given in the option list.) Starting from OTP 22, all options that can be given in the file can also be given in the option list.

For debugging of the compiler, or for pure curiosity, the intermediate code generated by each compiler pass can be inspected. To print a complete list of the options to produce list files, type `compile:options()` at the Erlang shell prompt. The options are printed in the order that the passes are executed. If more than one listing option is used, the one representing the earliest pass takes effect.

Unrecognized options are ignored.

Both `WarningList` and `ErrorList` have the following format:

```
[{FileName,[ErrorInfo]}].
```

`ErrorInfo` is described later in this section. The filename is included here, as the compiler uses the Erlang pre-processor `epp`, which allows the code to be included in other files. It is therefore important to know to **which** file the location of an error or a warning refers.

`forms(Forms)`

Is the same as `forms(Forms, [verbose,report_errors,report_warnings])`.

`forms(Forms, Options) -> CompRet`

Types:

```
Forms = forms()
forms() = [erl_parse:abstract_form | cerl:c_module()]
Options = [option()]
CompRet = BinRet | ErrRet
BinRet = {ok,ModuleName,BinaryOrCode} |
{ok,ModuleName,BinaryOrCode,Warnings}
ModuleName = module()
BinaryOrCode = binary() | term()
ErrRet = error | {error,Errors,Warnings}
Warnings = Errors = [{file:filename(), [{erl_anno:location() | 'none',
module(), term()}]}]
```

Analogous to `file/1`, but takes a list of forms (in either Erlang abstract or Core Erlang format representation) as first argument. Option `binary` is implicit, that is, no object code file is produced. For options that normally produce a listing file, such as 'E', the internal format for that compiler pass (an Erlang term, usually not a binary) is returned instead of a binary.

`format_error(ErrorDescriptor) -> chars()`

Types:

```
ErrorDescriptor = errordesc()
```

Uses an `ErrorDescriptor` and returns a deep list of characters that describes the error. This function is usually called implicitly when an `ErrorInfo` structure (described in section Error Information) is processed.

`output_generated(Options) -> true | false`

Types:

```
Options = [term()]
```

Determines whether the compiler generates a `beam` file with the given options. `true` means that a `beam` file is generated. `false` means that the compiler generates some listing file, returns a binary, or merely checks the syntax of the source code.

`noenv_file(File, Options) -> CompRet`

Works like `file/2`, except that the environment variable `ERL_COMPILER_OPTIONS` is not consulted.

`noenv_forms(Forms, Options) -> CompRet`

Works like `forms/2`, except that the environment variable `ERL_COMPILER_OPTIONS` is not consulted.

`noenv_output_generated(Options) -> true | false`

Types:

Options = `[term()]`

Works like `output_generated/1`, except that the environment variable `ERL_COMPILER_OPTIONS` is not consulted.

Default Compiler Options

The (host operating system) environment variable `ERL_COMPILER_OPTIONS` can be used to give default compiler options. Its value must be a valid Erlang term. If the value is a list, it is used as is. If it is not a list, it is put into a list.

The list is appended to any options given to `file/2`, `forms/2`, and `output_generated/2`. Use the alternative functions `noenv_file/2`, `noenv_forms/2`, or `noenv_output_generated/2` if you do not want the environment variable to be consulted, for example, if you are calling the compiler recursively from inside a parse transform.

The list can be retrieved with `env_compiler_options/0`.

Inlining

The compiler can do function inlining within an Erlang module. Inlining means that a call to a function is replaced with the function body with the arguments replaced with the actual values. The semantics are preserved, except if exceptions are generated in the inlined code. Exceptions are reported as occurring in the function the body was inlined into. Also, `function_clause` exceptions are converted to similar `case_clause` exceptions.

When a function is inlined, the original function is kept if it is exported (either by an explicit export or if the option `export_all` was given) or if not all calls to the function are inlined.

Inlining does not necessarily improve running time. For example, inlining can increase Beam stack use, which probably is detrimental to performance for recursive functions.

Inlining is never default. It must be explicitly enabled with a compiler option or a `-compile()` attribute in the source module.

To enable inlining, either use the option `inline` to let the compiler decide which functions to inline, or `{inline, [{Name, Arity}, ...]}` to have the compiler inline all calls to the given functions. If the option is given inside a `compile` directive in an Erlang module, `{Name, Arity}` can be written as `Name/Arity`.

Example of explicit inlining:

```
-compile({inline,[pi/0]}).  
  
pi() -> 3.1416.
```

Example of implicit inlining:

```
-compile(inline).
```

The option `{inline_size, Size}` controls how large functions that are allowed to be inlined. Default is 24, which keeps the size of the inlined code roughly the same as the un-inlined version (only relatively small functions are inlined).

Example:

```
%% Aggressive inlining - will increase code size.
-compile(inline).
-compile({inline_size,100}).
```

Inlining of List Functions

The compiler can also inline various list manipulation functions from the module `list` in `STDLIB`.

This feature must be explicitly enabled with a compiler option or a `-compile()` attribute in the source module.

To enable inlining of list functions, use option `inline_list_funcs`.

The following functions are inlined:

- `lists:all/2`
- `lists:any/2`
- `lists:foreach/2`
- `lists:map/2`
- `lists:flatmap/2`
- `lists:filter/2`
- `lists:foldl/3`
- `lists:foldr/3`
- `lists:mapfoldl/3`
- `lists:mapfoldr/3`

Parse Transformations

Parse transformations are used when a programmer wants to use Erlang syntax but with different semantics. The original Erlang code is then transformed into other Erlang code.

See `erl_id_trans(3)` for an example and an explanation of the function `parse_transform_info/0`.

Error Information

The `ErrorInfo` mentioned earlier is the standard `ErrorInfo` structure, which is returned from all I/O modules. It has the following format:

```
{ErrorLocation, Module, ErrorDescriptor}
```

`ErrorLocation` is the atom `none` if the error does not correspond to a specific location, for example, if the source file does not exist.

A string describing the error is obtained with the following call:

```
Module:format_error(ErrorDescriptor)
```

See Also

`epp(3)`, `erl_id_trans(3)`, `erl_lint(3)`, `beam_lib(3)`